

Weekly Learning Debrief

MIS410 — Data Analytics Certificate | Week of August 3, 2026

Topics: SQL time series functions, SQLAlchemy + MySQL, pandas resampling & interpolation, STL seasonal decomposition, matplotlib time series visualization

Reading: Time Series Analysis with SQL

Covered SQL's toolkit for working with time-based data: time zone conversions, date formatting and math using intervals, trending, percent-of-total calculations, indexing to a base period, rolling time windows (self-JOINS vs. window functions with a frame clause), cumulative values (YTD/QTd), and period-over-period comparisons using LAG to control for seasonality. Also reviewed a companion article covering the same ground in PostgreSQL, including RANK() and DENSE_RANK() for ranking values over time — not covered in the textbook chapter.

- Core throughline: raw timestamped data is noisy, and this whole toolkit exists to cut through that noise to find the real trend and seasonal pattern underneath.
- Watched a companion video on the four components of time series data — trend, seasonality, cyclic patterns, and irregular/residual noise — which mapped directly onto this week's SQL and Python material.

Assignment: SQL + Python Time Series Analysis

Built a full pipeline on a real provided retail sales dataset (200 daily transactions, Jan–Jul 2024): loaded the CSV into MySQL via SQLAlchemy, wrote five SQL queries (date range filtering, GROUP BY aggregation by day and product, WHERE filtering by store, and HAVING to filter on an aggregated SUM), then moved into Python for cleaning, resampling, interpolation, STL decomposition, and three matplotlib visualizations.

- **SQL:** BETWEEN for date ranges, GROUP BY for aggregation, and HAVING to filter on an already-aggregated value (SUM/COUNT) — something WHERE cannot do, since WHERE only filters raw rows before aggregation happens.
- **Cleaning:** checked for missing values (none found) and converted `transaction_date` from text to a real datetime type using `pd.to_datetime()` — required before any resampling would work.
- **Resampling:** used `.groupby("product_id").resample("W")` to get weekly sales totals per product — grouping happens first, then each group is resampled into weekly chunks independently.
- **Interpolation:** since the real dataset had no missing days, manually created a test gap to demonstrate `.interpolate(method="linear")` — correctly estimated the missing value as the exact midpoint between its two neighboring days, confirming the “straight line between two known points” logic.
- **STL decomposition:** used `statsmodels.tsa.seasonal.STL` to split daily sales into trend, seasonal, and residual components with `period=7` (weekly seasonality). Found a declining overall trend and a recurring weekly pattern.
- **Visualization:** time series plot of total sales, a by-product weekly trend plot (using `.unstack()` to pivot grouped data into one column per product), and the STL decomposition plot — all three required visualizations completed.
- **Key finding:** Product 104 stood out as consistently higher and more volatile than the other four products week over week.

SQL & Python Time Series Toolkit

Method	What it does
<code>pd.to_datetime(col)</code>	Converts a text column into a real date type
<code>.set_index("date_col")</code>	Makes the date column the row index, required for resampling
<code>.resample("W")</code>	Regroups data into a new time frequency (weekly, monthly, etc.)

<code>.interpolate(method="linear")</code>	Estimates missing values along a straight line between known points
<code>STL(series, period=7)</code>	Splits a time series into trend, seasonal, and residual components
<code>.unstack(level="col")</code>	Pivots grouped/stacked data into wide format, one column per group
<code>BETWEEN ... AND ...</code>	SQL: filters rows within a date or value range
<code>GROUP BY</code>	SQL: groups rows for aggregation
<code>HAVING</code>	SQL: filters groups after aggregation (WHERE can't filter on SUM/COUNT results)

Debugging Patterns Worth Remembering

- **Kernel restarts wipe all variables.** Old output text stays visible on screen even after a restart, which can make a notebook look “fine” when it's actually empty underneath. Fix: Kernel → Restart Kernel and Run All Cells, to rebuild everything top to bottom in order.
- **Raw cells don't execute anything** — not as Python, not as Markdown. A third cell type beyond Code/Markdown; if a cell produces zero output and zero error, check whether it's accidentally set to Raw.
- **Multi-line SQL in Python always needs the wrapper:** `query = """ SQL HERE """`, then `pd.read_sql(query, con=engine)`. Raw SQL pasted directly into a code cell will always throw a `SyntaxError` — Python doesn't know SQL.
- **Imports don't carry over between notebooks.** Each notebook has its own separate memory — `import matplotlib.pyplot as plt` has to be re-run in every new notebook, even if it was already run in a different one earlier.
- **File paths across programs:** a file saved by one program (or uploaded from Documents/Downloads) isn't automatically visible to another unless the exact same folder is used, or a full path is given. “Copy as path” in File Explorer removes the guesswork.

Biggest “Aha” Moments

- `GROUP BY` filters raw rows before aggregation (via `WHERE`); `HAVING` filters the aggregated groups after the fact — two different filtering stages, not interchangeable.
- Interpolation is an estimate, not a real recorded value — worth flagging as such rather than treating it the same as actual data, especially in financial contexts.
- Multi-column `.groupby()` in pandas needs the columns wrapped in a list: `.groupby(["col1", "col2"])`, not passed as separate arguments.
- A chart with many overlapping time series lines isn't necessarily bad design — sometimes visual density is an unavoidable tradeoff of showing several categories on one chart at once, worth noting as an observation rather than a flaw to fix.
- Recognizing a concept while reading (“oh, that's the rolling window thing”) comes well before being able to reproduce the exact syntax from memory — normal, expected gap, not a sign of not understanding it.

Next steps: bring this same format back each week to track growth toward the Mod 8 goal of working independently.